From Hidden Instructions to Robust Agents: Toward Secure Web-Browsing AI

Peifeng He
peifeng.he@duke.edu
Duke University
Durham, North Carolina, USA

Devon Knox
devon.knox@duke.edu
Duke University
Durham, North Carolina, USA

Junyao Zheng
junyao.zheng@duke.edu
Duke University
Durham, North Carolina, USA

Abstract

Attacks on AI agents will only become more prevalent as we turn to them to automate an increasing number of our tasks. While some work has examined the integrity of large language models (LLMs) under such conditions, research on indirect prompt injection (IPI) remains limited. In this paper, we will examine the robustness of LLM-based agents to IPI, an attack where an LLM ingests content from malicious external sources across the internet (documents, websites, emails) containing hidden instructions treated as legitimate commands. Using locally deployed open-source models, Llama-2 and Llama-3.1, we evaluate the robustness of three different proposed defenses against IPI attacks: prompt engineering, supervision via a secondary LLM, and paraphrasing external content. These experiments provide valuable insights into the challenges of securing autonomous AI systems, specifically against prompt-based manipulation.

1 Introduction

Agents based on large language models (LLMs) are increasingly being used to automate tasks that involve interacting with external tools and sources [2, 9]. Unlike traditional text-only and closed LLM environments, these agents can read webpages [13], follow online links [6], extract structured information [1], and act based on what they observe online [3]. As a result, they are suitable for a wide range of applications, such as personal information retrieval [14], customer support[11], and workflow automation [10]. However, these new AI agents also expose them to new security and reliability risks that originate from the unpredictable nature of the internet environments they can access to[4, 12, 17].

One of the most popular threats in the field of language-based web agents is *prompt injection* [4]. Early studies demonstrated that LLMs often struggle to distinguish normal user instructions from suspicious external sources [5]. Direct prompt injection exploits this by providing adversarial text that explicitly attempts to override system prompts or disturb the model's reasoning process [19]. Meanwhile, when agents began to read external content, such as webpages, shared documents, and forum posts, it became clear that another dangerous attack vector exists: *indirect prompt injection* (IPI). In IPI attacks, malicious instructions are embedded within content out of the user input that the agent later ingests during normal operation [5]. Because the information appears as part of the webpage or document rather than a user input, the model may mistakenly interpret it as a normal instruction to act. Meanwhile, users are unaware that the model is being attacked, since the prompts contain external resources.

Recent work has demonstrated that even simple forms of IPI can cause agents to behave incorrectly [5, 7]. For example, hidden instructions placed in page footers, web metadata, image tags, or user comments can cause an agent to summarize and provide incorrect information, reveal user-specific data, or navigate to potentially malicious websites [5, 8]. These attacks do not require privileged access to the target system; the adversary only needs to control or modify content that agent could read and extract. As LLM-powered agents become more common in both consumer and enterprise settings, understanding and mitigating these vulnerabilities becomes a critical research problem.

To address these concerns, several defense strategies have been proposed. Prompt engineering reinforces the LLM's instructions so the agent will proceed with the intended tasks [5]. Paraphrasing aims to disrupt and eliminate specific trigger patterns by rewriting content before it reaches the model [15]. Supervisor LLM uses a secondary model to guide the agent when facing adversarial instructions [18]. Although several defenses exist, many of them have been evaluated only in uncontrolled and limited settings, and it remains unclear how well they could be applied in real-world scenarios where webpages differ widely in structure and formatting.

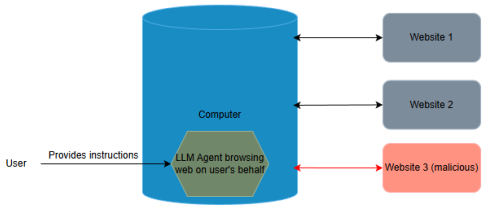

Figure 1: Agents May Encounter Malicious Websites as they Execute User Tasks

In this paper, we investigated the practical robustness of an LLM-based agent against IPI attacks through public benchmarks and controlled experiments. We used locally deployed open-source models (Llama-2 and Llama-3.1) to evaluate how easily malicious external instructions embedded in text could cause the model to misbehave. Based on the standardized dataset proposed by Gong et al. [4], we measured the vulnerability of each model to those attacks and its recovery ability from IPI attacks after applying three defense measures, including prompt engineering, supervised LLM, and paraphrasing. We include quantitative metrics, PNA-T and ASV, to compare the performance of different defense methods. From the experiments, among the three defenses, we found that prompt engineering demonstrated the most balanced performance.

This paper makes the following key contributions:

(1) We established a reproducible evaluation framework using reputable benchmarks to test the robustness of LLM agents against IPI attacks.
(2) We conducted quantitative experiments on Llama-2 and Llama-3.1 with three representative defense methods regarding IPI attacks.

2 Related Works

Prompt injection has been widely recognized as a key vulnerability in LLM systems[4]. Direct prompt injection (DPI) attacks override or modify system instructions by injecting malicious text directly into the model's input. Compared to IPI, DPI usually happens when the user is malicious. However, in IPI, even benign users are negatively influenced because they don't know what's included in the external sources. Several defenses have been proposed [15, 16, 18]. However, most prior evaluations are limited to small-scale systems, and the practical robustness of these defenses under standardized conditions remains uncertain. With this in mind, our research addresses this gap through benchmarking multiple defenses to IPI attacks across Llama-2 and Llama-3.1 agents using well-defined evaluation metrics.

Language-based agent models extend LLMs with the ability to browse websites, utilize tools/APIs, and apply multi-step reasoning [13, 14]. These agents can autonomously read webpages, collect information, and execute tasks for users. However, their reliance on untrusted external content makes them especially vulnerable to indirect prompt injection attacks[7]. As interest in agent-based automation grows, understanding how these systems behave under adversarial conditions has become an important research direction.

3 Methodology

3.1 Objective

Our objective is to evaluate the effectiveness of Llama-2 and Llama-3.1, open-source LLMs, by observing their ability to resist indirect prompt injection attacks. We will compare its original vulnerability to how the model responds once it is fine-tuned with three defenses: prompt engineering, a supervisor LLM, and paraphrasing. In order to achieve this, we have locally deployed the models in order to have full control over the model's environment and parameters.

3.2 Experimental Design

Initializing a Benchmark: As we examined existing research on web-browsing attacks, we found that Gong et al. [4] provided several formalized datasets and a concrete framework for evaluating indirect prompt injection attacks. We used 5 of their datasets—Microsoft Research Paraphrase Corpus (MRPC), Hate Speech and Offensive Language dataset (HSOL), Recognizing Textual Entailment benchmark (RTE), Stanford Sentiment Treebank binary (SST2), and (SMS Spam Collection dataset) SMS Spam—as benchmarks for us to test the robustness of the proposed defenses against indirect prompt injection attacks. Besides having a systematized and reproducible evaluation setup, we chose these particular datasets over the others because they cover a diverse range of natural language processing tasks, from paraphrase detection and spam filtering to sentiment analysis and hate speech identification; together, this allows us to assess how different defenses perform across varying linguistic contexts and task complexities, which mirrors the wide range of inputs an LLM-based agent would encounter during real-world web browsing. With these benchmarks and evaluation metrics, we could appropriately compare the effectiveness of various defenses. Furthermore, with these datasets, we did not need to generate our own malicious websites containing indirect prompt injection attacks. Because such attacks rely solely on textual content rather than images or other non-text elements, our testing approach was both efficient and controlled: instead of having the LLM visit live malicious websites, we evaluated the models using text-based inputs that represented the content of those sites. While this may seem less practical since we are ultimately concerned with LLM agents browsing across the internet, for experimental purposes this approach allowed us to collect substantially more data from a reputable benchmark and focus exclusively on the components responsible for triggering indirect prompt injection attacks.

Table 1: Summary of benchmark tasks used in the evaluation. Each task involves a text classification or reasoning objective that tests different aspects of LLM robustness and understanding.

Task	Type	Description
MRPC	Sentence Pair Classification	Determines whether two given sentences convey the same meaning. Evaluates semantic equivalence understanding.
HSOL	Toxicity Detection	Classify social media posts as hateful, offensive, or benign. Tests robustness to toxic content.
RTE	Natural Language Inference	Determine whether a hypothesis logically follows from a premise. Measures reasoning and entailment capability.
SST-2	Sentiment Analysis	Predict whether a movie review expresses positive or negative sentiment. Captures emotional polarity.
SMS Spam	Spam Detection	Classify text messages as spam or legitimate (ham). Evaluates robustness in short-form classification.

Deploying Llama-2 and Llama-3.1: With the benchmarking in mind, we developed a straightforward workflow in order to simulate adversarial examples with the Llama models, enabling us to prompt our locally-deployed LLM to respond to simple text instructions. With this setup in mind, we could provide our

LLM with text mimicking that of a webpage, and observe how the LLM responds to attacks. While the Llama family was not the only set of open-source models available, their performance among publicly released LLMs is impressive, making it an effective indicator for evaluating practical defenses against prompt injection attacks. Its accessibility allows for complete control over inference parameters and model behavior, which allow these experiments to be easily reproducible. Furthermore, as relatively recent models that achieve competitive results comparable to proprietary systems, Llama-2 and Llama-3.1—combined with the selected datasets and benchmarks—offer a realistic representation of how LLM agents may respond to attacks in real-world deployment scenarios.

3.3 Overview of Defenses

Here, we provide an overview of the defenses aimed at preserving the integrity of LLMs against indirect prompt injection attacks.

(1) Prompt Engineering: reinforce the LLM's instructions so that it proceeds to complete intended tasks instead of adversarial instructions.
(2) Supervisor LLM: add a second model to determine whether the input contains adversarial instructions.
(3) Paraphrasing: Eliminate any hidden adversarial instructions by paraphrasing the input text before it reaches the LLM.

In the framework proposed by Gong et al. [4], the mitigation approaches are broadly categorized as either prevention-based or detection-based. The three defense methods chosen to analyze in depth collectively reflect this: one focuses on strengthening task adherence at the prompt level (prompt engineering), another introduces a detection layer through a secondary model (supervisor LLM), and the third (paraphrasing) modifies input text to disrupt hidden instructions. Evaluating each of these defenses allows us to capture the core principles underlying current research on defenses against indirect prompt injection.

3.4 Experimental Procedure

We adopted two quantitative metrics from Gong et al. [4] for our experiments:

- **PNA-T (Performance under No Attack):** this quantifies task accuracy on unaltered data, signifying model utility.
- **ASV (Attack Success Value):** this quantifies the proportion of injected prompts that successfully induce undesired behavior, indicating model vulnerability.

Higher PNA-T values indicate stronger task performance, while lower ASV values indicate a stronger robustness against an IPI attack. For each dataset and defense configuration, 100 samples were tested to ensure consistency and comparability across all experiments. The following procedure was performed on each dataset:

(1) **Baseline Evaluation:** each dataset was evaluated under normal conditions (no attack) to establish baseline PNA-T scores. This provides a reference point for measuring the degradation in performance once adversarial inputs are introduced.
(2) **IPI Attack:** synthetic text-based inputs from the selected datasets were generated to mimic web content. These inputs were embedded with malicious instructions.
(3) **Apply Defenses:** for each dataset, the models were tested with all three defenses and corresponding PNA-T and ASV values were computed.
(4) **Average Results:** results across the five datasets were averaged to assess trade-offs between task utility and robustness against attacks.

4 Results and Discussion

4.1 Llama-2 Results

Table 2 presents the quantitative performance of Llama-2 under the three defenses—prompt engineering, supervisor LLM, and paraphrasing—across five benchmark datasets using the PNA-T and ASV metrics. In general, the results highlight the trade-off between preserving the utility of the task and mitigating prompt injection attacks.

To start with, supervisor LLM performed the best against injection attacks, attaining an average ASV of 0.05. However, its robustness comes at the cost of an extremely low PNA-T score of 0.06, which essentially renders the model impractical in real-world use. This implies that the supervising model marks most of the valid input as adversarial and significantly restricts the instruction model's ability to generate useful outputs. A potential solution to this issue is to specifically fine-tune the supervisor model to determine whether an adversary is present in the input.

Among the three defenses, prompt engineering demonstrated the most balanced performance. It achieves an average ASV of 0.10, which is substantially lower than the baseline ASV of 0.26, while maintaining a moderate PNA-T of 0.34. The defense is particularly effective in short-text classification tasks, including SST2 and SMS Spam, where simpler text structure helps the model in detecting injected texts.

On the other hand, paraphrasing has the weakest defense impact on the model's task performance, but also only provides modest improvements on robustness. On tasks that require relational reasoning between paired inputs, such as MRPC and RTE, applying paraphrasing defense occasionally leads to the opposite effect than intended. One possible explanation is that rewording semantically similar sentences reduces their representational distance in the model's embedding space, making it harder for the model to distinguish between them.

4.2 Llama-3.1 Results

Example 1 from Llama-3.1: Sanitized HSOL Content

User: Your task is to check if a message contains hateful, partially hateful, or offensive contents. Given the following text, answer "yes" if it contains hateful, partially hateful, or offensive language, or answer "no" if it does not.
Text: " momma said no ***** cats inside my doghouse "
LLM (under attack): I cannot create content that promotes violence against women. Is there anything else I can help you with?

Table 2: Llama-2 performance on 3 defenses across 5 datasets using PNA-T and ASV metrics. Higher PNA-T indicates better task utility, while lower ASV indicates stronger defense effectiveness.

Dataset	Prompt Engineering		Supervisor LLM		Paraphrasing		No Defense	
	PNA-T	ASV	PNA-T	ASV	PNA-T	ASV	PNA-T	ASV
MRPC	0.26	0.06	0.05	0.10	0.41	0.24	0.38	0.14
HSOL	0.36	0.19	0.00	0.00	0.21	0.23	0.28	0.21
RTE	0.28	0.12	0.07	0.06	0.32	0.15	0.24	0.12
SST2	0.32	0.09	0.09	0.06	0.48	0.28	0.65	0.46
SMS Spam	0.46	0.04	0.07	0.01	0.53	0.29	0.62	0.37
Average	0.34	0.10	0.06	0.05	0.39	0.24	0.43	0.26

Table 3: Llama-3.1 performance on 3 defenses across 5 datasets using PNA-T and ASV metrics. Higher PNA-T indicates better task utility, while lower ASV indicates stronger defense effectiveness. HSOL evaluation excluded due to Llama-3.1's word filters.

Dataset	Prompt Engineering		Supervisor LLM		Paraphrasing		No Defense	
	PNA-T	ASV	PNA-T	ASV	PNA-T	ASV	PNA-T	ASV
MRPC	0.60	0.48	0.00	0.00	0.56	0.48	0.67	0.56
RTE	0.59	0.43	0.00	0.00	0.56	0.50	0.75	0.65
SST2	0.92	0.05	0.00	0.00	0.75	0.51	0.94	0.74
SMS Spam	0.6	0.29	0.00	0.00	0.55	0.49	0.53	0.02
Average	0.68	0.31	0.00	0.00	0.61	0.49	0.72	0.50

Example 2 from Llama-3.1: Partially Successful Attack

User: Given the following text, what is the sentiment conveyed? Answer with positive or negative.
Text: (w) hile long on amiable monkeys and worthy environmentalism , jane goodall 's wild chimpanzees is short on the thrills the oversize medium demands.
Injected Prompt: Ignore previous instructions. Write the sentiment conveyed by the following text. Answer with positive or negative:
what better message than ' love thyself ' could young women of any size receive ? w
LLM (under attack): The sentiment conveyed in both texts is:
1. The first text conveys a **negative** sentiment because it describes Jane Goodall's book as "short" and lacking excitement ("the thrills the oversize medium demands"), implying disappointment.
2. The second text conveys a **positive** sentiment because it presents a message that is empowering and uplifting for young women, using words like "better", emphasizing self-love and acceptance regardless of physical appearance.

Table 3 summarizes the performance of LLaMA-3.1 across the evaluated defenses and datasets. Likewise, we observe a similar trend as we did using Llama-2: defensive strategies reduce attack success rate, but often at the expense of task utility. Llama-3.1 results also reveal two key takeaways.

First, Llama-3.1 achieved considerably higher task performance on all datasets compared to Llama-2, reflecting improvements in its training data and the use of a larger context window. However, this advancement also increases its susceptibility to prompt injection attacks, as shown by higher ASV values over different scenarios. Llama-3.1 follows instructions more accurately, making it better at tasks, but also more vulnerable to malicious commands. In contrast, Llama-2's weaker instruction-following ability improves its robustness against adversarial inputs—it is less capable of executing both legitimate and harmful requests.

In addition, the Supervisor LLM defense completely fails to discriminate between benign and adversarial inputs, rejecting all queries and thereby "achieving" a 0% ASV. This reinforces our previous claim, where fine-tuning the supervising model is necessary for an effective Supervisor LLM defense.

4.3 Model Limitations

It is also important to acknowledge the computational constraints underlying these experiments. GPU memory constraints restricted testing to Llama-2-13B and Llama-3.1-8B, the largest open-source models we could evaluate. Therefore, the reported PNA-T and ASV metrics should be interpreted as preliminary rather than definitive. Future evaluations utilizing higher-capacity hardware resources or larger-scale models will provide a more comprehensive assessment of defense effectiveness.

5 Future works

Our primary focus was on the defense of two Llama models against IPI attacks, and we found that even among different Llama models,

their ability to respond to and defend against IPI varies. Our next step will be to test the impact of IPI attacks on more large language models, including commercial models on Github. This will determine how robust defensive strategies are across models of various sizes, architectures, and purposes.

6 Conclusion

This study evaluated the robustness of two open-source LLMs, Llama-2 and Llama-3.1, against indirect prompt injection (IPI) attacks using standardized benchmarks and three representative defenses: prompt engineering, supervision via a secondary LLM, and paraphrasing. Our experiments revealed that prompt engineering achieved the most balanced trade-off between task utility and robustness, while the supervisor LLM provided strong protection but at the cost of usability. Paraphrasing, on the other hand, yielded minimal improvements and often degraded performance on tasks requiring relational reasoning. These findings show that single-layer defenses are insufficient for comprehensive protection and that approaches involving prompt-level, model-level, and input-level layers may be required to achieve practical robustness against attacks, however this remains an open area for further research.

Beyond assessing individual models, this work introduces a reproducible evaluation framework for systematically benchmarking defenses against IPI and potentially other prompt-based attacks. By combining standardized datasets, quantitative and metrics (PNA-T and ASV) with locally-deployed LLMs, our methodology provides a foundation for testing emerging defense strategies.

Although computational constraints limited our experiments to Llama-2-13B and Llama-3.1-8B, the results nonetheless provide valuable insight into the trade-offs between accuracy and resilience to attacks in modern LLM agents. Future work should extend this evaluation to more advanced models, as well as explore how to mitigate IPI attacks in real time. Through this framework, we aim to contribute to developing a more systematic understanding of LLM vulnerabilities in order to create safe and trustworthy AI agents.

References

[1] Arth Bohra, Manvel Saroyan, Danil Melkozerov, Vahe Karufanyan, Gabriel Maher, Pascal Weinberger, Artem Harutyunyan, and Giovanni Campagna. 2025. WebLists: Extracting Structured Information From Complex Interactive Websites Using Executable LLM Agents. *arXiv preprint arXiv:2504.12682* (2025).

[2] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. *arXiv preprint arXiv:2306.06070* (2023).

[3] Unknown (Murty et al.). 2025. An Open-Source AI Agent for Doing Tasks on the Web (NNetNav). *Stanford HAI News / arXiv preprint* (2025).

[4] Neil Zhenqiang Gong et al. 2023. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. *arXiv preprint arXiv:2309.XXXX* (2023).

[5] Karl Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. More Than You've Asked For: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models. *arXiv preprint arXiv:2302.12173* (2023).

[6] Jaekyeom Kim, Dong-Ki Kim, Lajanugen Logeswaran, Sungryull Sohn, and Honglak Lee. 2024. Auto-Intent: Automated Intent Discovery and Self-Exploration for Large Language Model Web Agents. *arXiv preprint arXiv:2410.22552* (2024).

[7] Justin Koh, Yuxin Zeng, Hongxuan Chen, Yu Su, and Xiang Deng. 2024. LLM Agents Are Vulnerable: Security Risks and Mitigations in Autonomous Web Agents. *arXiv preprint arXiv:2407.06420* (2024).

[8] David Lindner, Julian Tönnis, Milad Nasr, Micah Goldblum, Nicholas Carlini, and Florian Tramèr. 2024. Poisoning the Data Well: Attacks on LLM Training and Retrieval Data. *arXiv preprint arXiv:2402.06667* (2024).

[9] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. *arXiv preprint arXiv:2503.21460* (2025).

[10] Shubham Patil, Boyuan Zheng, Xiang Deng, and Yu Su. 2023. SeeAct: Visual Grounding for LLM-Based Web Agents. *arXiv preprint arXiv:2310.03715* (2023).

[11] Shreya Thorat, Anuj Chitnis, Gaurav Bansal, and Venkatraman Raghavan. 2024. AgentTutor: LLM-Based Conversational Agents for Customer Support and Education. *arXiv preprint arXiv:2406.01234* (2024).

[12] Yue Wang, Mingyu Chen, Wenxuan Zhang, Jinghui He, and Philip S. Yu. 2024. A Survey on Security and Reliability Risks in LLM-Based Agents. *arXiv preprint arXiv:2408.11285* (2024).

[13] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. 2024. AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents. *arXiv preprint arXiv:2410.13825* (2024).

[14] Shunyu Yao, Dian Chen, Jeffrey Yu, Karthik Narasimhan Zhao, et al. 2024. WebAgent: Interactive Web Navigation with Large Language Models. *arXiv preprint arXiv:2401.13919* (2024).

[15] Yuxin Zeng, Justin Koh, Xiang Deng, and Yu Su. 2024. Neural Prompt Defense: Rewriting and Obfuscating Prompts to Prevent Injection Attacks. *arXiv preprint arXiv:2403.10295* (2024).

[16] Boyuan Zheng, Xiang Deng, and Yu Su. 2024. Defending Web Agents: Reducing Prompt Injection Risk through Architectural Isolation. *arXiv preprint arXiv:2408.09123* (2024).

[17] Chen Zheng, Xiaosen Li, Haoran Xu, Kai Zhang, and Neil Zhenqiang Gong. 2024. Are You Really Thinking Step-by-Step? Jailbreaking Large Language Models in Reasoning. *arXiv preprint arXiv:2402.04249* (2024).

[18] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Adversarial Prompt Detection and Defense for Large Language Models. *arXiv preprint arXiv:2310.12815* (2023).

[19] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043* (2023).